



浙江大学爱丁堡大学联合学院

ZJU-UoE Institute

Introduction to Programming 1

IB1 1, Lecture 4.1

Dr Rob Young – robert.young@ed.ac.uk

Semester 2, 2025/26

How do you tell a computer what to do?



Principles of programming



Learning objectives

After this double lecture, you should be able to:

- Plan a project using pseudocode.
- Explain how variables work within a piece of code.
- Explain the concept of loops.
- Understand and use Booleans.

Outline

1. What does programming even mean?
2. Planning a project using pseudocode.
3. Variables.

How to run 800 m? (Teach a computer)



How to run 800 m? (Teach a computer)

```
# Function: Run 800m
#     Repeat:
#         Run
#         Check if there is a bend
#         If yes: Turn
#         If no: Run straight ahead
#         Count laps
#         How many laps completed?
#             If less than 2: Keep running
#             If 2: DONE!
```

How to run 800 m? (Teach a computer)

```
# Function: Run
#     Repeat:
#         Push off from right foot
#         Land on left foot
#         Move right foot forward
#         Push off from left foot
#         Land on right foot
#         Move left foot forward
```


Why is this difficult?



Why is this difficult?

Computers (but not dogs) are stupid!



What kind of things do we need?



- Define re-usable functions.
- Count and keep track of numbers.
- Compare things to other things.
- Include decision points
- Repeat things until something happens.

What kind of things do we need?

- Define re-usable functions.

Run

- Count and keep track of numbers.

Laps completed

- Compare things to other things.

Have 2 laps been completed?

- Include decision points

Turn if there is a bend, run straight ahead if there isn't

- Repeat things until something happens

Run until 800m are complete

Outline

1. What does programming even mean?
2. Planning a project using pseudocode.
3. Variables.

Pseudocode: What? Why?



```

28     expires = new Date().toLocaleDateString()
29     document.cookie = "expires=" + expires + "; path=/";
30 }
31 function validateForm() {
32     var x = document.forms["myForm"]["fname"].value;
33     if (x == "") {
34         alert("Name must be filled out");
35         return false;
36     }
37 }
38 var marker = new toggle.secure.Marker({image: log, position: 'top',
39     marker.addListener('click', function() {
40         infowindow.open(log, marker);
41     });
42 }
43 <form name="myForm" action="/action_page.php"
44     Name: <input type="text" name="fname">
45     <input type="submit" value="Submit">

```

- Translating human thoughts into programming code is difficult.

Pseudocode: What? Why?



```

28 expires = "expires=" + (new Date(1000000000000)).toUTCString()
29 document.cookie = name + "=" + value + "; expires=" + expires + "; path=" + "/"
30 }
31 function validateForm() {
32   var x = document.forms["myForm"]["name"].value
33   if (x == "") {
34     alert("Name must be filled out");
35     return false;
36   }
37 }
38 var marker = new toogle.secure.Marker({image: log, position: 'top',
39   marker.addListener('click', function() {
40     infowindow.open(log, marker);
41   });
42   });
43 <form name="myForm" action="/action_page.php">
44   Name: <input type="text" name="name">
45   <input type="submit" value="Submit">

```

- Translating human thoughts into programming code is difficult.
- Pseudocode provides an “intermediate step”.

Example

Human language

Run around the track for two laps

Example

Human language

Run around the track for two laps

Pseudocode

Repeat :

Run

How many laps completed?

If less than 2: Keep running

If 2: DONE!

Example

Human language

Run around the track for two laps

Pseudocode

Repeat:

```
#           Run
#           How many laps completed?
#           If less than 2: Keep running
#           If 2: DONE!
```

Code

```
for laps in range(1,3):
    running()
    if (laps == 2):
        done()
```

Pseudocode: What? Why? How?



```
28 expires = new Date().getTime() + (7 * 24 * 60 * 60 * 1000);
29 document.cookie = "expires=" + expires + ";path=/";
30 }
31 function validateForm() {
32     var x = document.forms["myForm"]["fname"].value;
33     if (x == "") {
34         alert("Name must be filled out");
35         return false;
36     }
37 }
38 var marker = new toogle.secure.Marker({image: log, position: verify});
39 marker.addListener('click', function() {
40     infowindow.open(log, marker);
41 });
42 }
43 <form name="myForm" action="/action_page.php" method="text">
44     Name: <input type="text" name="fname">
45     <input type="submit" value="Submit">
```

- Translating human thoughts into programming code is difficult.
- Pseudocode provides an “intermediate step”.
- There is no specified syntax for pseudocode – it is up to the individual programmer!

Why the hashtags?

- Programming languages have symbols for "comments".
- In Python (and R): #
- If a line starts with a comment symbol, it is not read when the programme is interpreted.
- If comments are used, pseudocode and code can be used in the same document.

```
# Repeat
    for laps in range(1,3):
#         Run
            running()
#         How many laps completed?
#             If less than 2: Keep running
#             If 2: DONE!
            if (laps == 2):
                done()
```

What are the advantages of using pseudocode?

What are the advantages of using pseudocode?

- Helps with planning out the steps needed for a project.
- Helps remind yourself what you were going to do next.
- Can help assign coding tasks in collaborative projects.
- Helps explain and document code.

Outline

1. What does programming even mean?
2. Planning a project using pseudocode.
3. Variables.

Variables

- Variables are objects that can store values – they are a way of giving names to things (e.g. distance run so far).
- Unlike a “variable” in physics, they can be constants (or not).
- Variable assignment:

`x = 5`

But *not*:

`5 = x`

- Variables can be of different types (e.g. number, string). Python is quite clever at automatically recognising variable type.

Number variables

Numbers

- Number variables store numbers. Once a variable has been assigned it can be used the same way as the number would be used.

5+7

Number variables

Numbers

- Number variables store numbers. Once a variable has been assigned it can be used the same way as the number would be used.

5+7

12

Number variables

Numbers

- Number variables store numbers. Once a variable has been assigned it can be used the same way as the number would be used.

5+7

12

bla = 5

bla+7

Number variables

Numbers

- Number variables store numbers. Once a variable has been assigned it can be used the same way as the number would be used.

5+7

12

bla = 5

bla+7

12

Number variables

Numbers

- Variables can be incremented to use as counters.

```
x = 2
```

```
x = x+1
```

```
x
```

Number variables

Numbers

- Variables can be incremented to use as counters.

```
x = 2
```

```
x = x+1
```

```
x
```

```
3
```

Number variables

Numbers

- Variables can be incremented to use as counters.

```
x = 2
```

```
x = x+1
```

```
x
```

```
3
```

```
y = 1
```

```
y += 1
```

Number variables

Numbers

- Variables can be incremented to use as counters.

```
x = 2
```

```
x = x+1
```

```
x
```

3

```
y = 1
```

```
y += 1
```

2

Number variables

Example 1

$a = 12$

$b = 2$

What is $a+b$, $a-b$, $a*(-b)$, a/b , $a**b$, $a\%b$?

Number variables

Example 1

$a = 12$

$b = 2$

What is $a+b$ (14), $a-b$ (10), $a*(-b)$ (-24), a/b (6), $a**b$ (144), $a\%b$ (0)?

Number variables

Example 1

$a = 12$

$b = 2$

What is $a+b$ (14), $a-b$ (10), $a*(-b)$ (-24), a/b (6), $a**b$ (144), $a\%b$ (0)?

Example 2

$x = 1$

$y = 2$

$x = x*y$

$z = x$

$x = x*y$

At the end of this, what is x ? What is y ? What is z ?

Number variables

Example 1

$a = 12$

$b = 2$

What is $a+b$ (14), $a-b$ (10), $a*(-b)$ (-24), a/b (6), $a**b$ (144), $a\%b$ (0)?

Example 2

$x = 1$

$y = 2$

$x = x*y$

$z = x$

$x = x*y$

At the end of this, what is x (4)? What is y (2)? What is z (2)?

String variables

Strings

- Strings are sequences of letters, used to store text
- String variables are specified using single or double quotation marks

```
a = "hello "
```

```
B = 'world!'
```

- String variables can be combined using +

```
a+b
```

```
'hello world!'
```

String variables

Strings

- Strings are sequences of letters, used to store text
- String variables are specified using single or double quotation marks

```
a = "hello "
```

```
B = 'world!'
```

- String variables can be combined using +

```
a+b
```

```
'hello world!'
```

Example 1

```
x = "Hello, my name is "
```

```
y = "Rob"
```

What is x+y? How can you change it to say your name instead?

String variables

Example 2

```
c = "four"
```

```
d = "two"
```

What is `c+2*d`?

String variables

Example 2

```
c = "four"
```

```
d = "two"
```

What is `c+2*d`? `fourtwotwo`

String variables

Example 2

`c = "four"`

`d = "two"`

What is `c+2*d`?

Example 3

`i = "4"`

`j = "2"`

What is `i+j`?

String variables

Example 2

`c = "four"`

`d = "two"`

What is `c+2*d`?

Example 3

`i = "4"`

`j = "2"`

What is `i+j`? 42

Next...



- Define re-usable functions.
- Count and keep track of numbers.
- Compare things to other things.
- Include decision points
- Repeat things until something happens

Next...



- Define re-usable functions.
- Count and keep track of numbers.
- Compare things to other things.
- Include decision points.
- Repeat things until something happens.